

Học Máy (Machine Learning)

Chương 7: Học tổ hợp và Rừng ngẫu nhiên

Giảng viên: TS. Trần Thành Thắng

Đại học Đông Á

August 5, 2026

Nội dung Chương trình

- 1 Học tổ hợp (Ensemble Learning)
- 2 Bộ phân loại biểu quyết
- 3 Túi hóa (Bagging) và Dán nhãn (Pasting)
- 4 Rừng ngẫu nhiên (Random Forests)
- 5 Tăng cường (Boosting)
- 6 Phân lớp xếp chồng (Stacking)
- 7 Tổng kết Chương 7

- **Định nghĩa:** Một phương pháp kết hợp các dự đoán của một nhóm các bộ dự đoán (bộ phân loại hoặc bộ hồi quy) để đạt được kết quả dự đoán tốt hơn so với bất kỳ bộ dự đoán riêng lẻ nào.
- Một nhóm các bộ dự đoán được gọi là **một tập hợp (ensemble)**.
- Phương pháp học này được gọi là **học tổ hợp (ensemble learning)**.
- Thuật toán sử dụng kỹ thuật này gọi là **phương pháp tổ hợp (ensemble method)**.

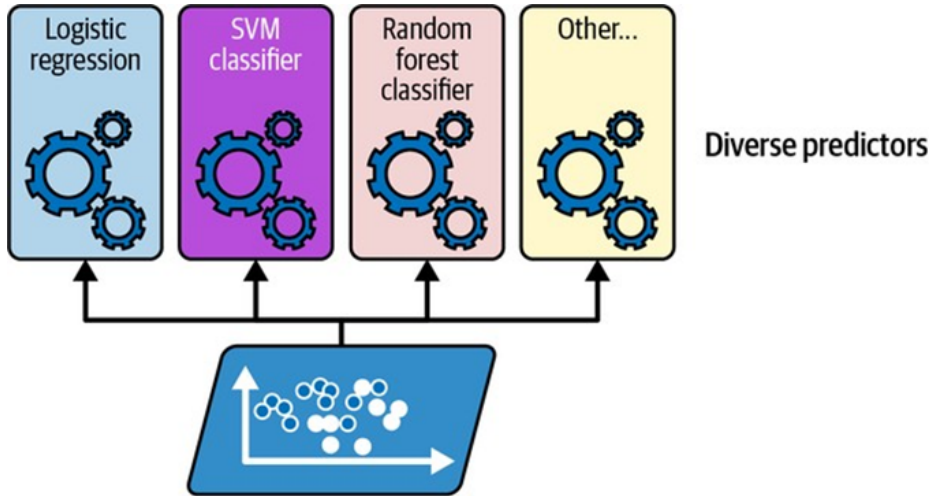
Sự khôn ngoan của đám đông

- Giả sử bạn đặt một câu hỏi phức tạp cho hàng nghìn người ngẫu nhiên, sau đó tổng hợp câu trả lời của họ.
- Trong nhiều trường hợp, câu trả lời tổng hợp này tốt hơn câu trả lời của một chuyên gia → **Sự khôn ngoan của đám đông**.
- **Ví dụ:** Huấn luyện một nhóm các bộ phân loại cây quyết định trên các tập con ngẫu nhiên khác nhau của tập huấn luyện → **Rừng ngẫu nhiên (Random Forest)**.
- Thường được áp dụng gần cuối một dự án Machine Learning để đẩy tối đa độ chính xác của hệ thống.

Bộ phân loại biểu quyết (Voting Classifiers)

- Bắt đầu bằng việc huấn luyện một vài bộ phân loại độc lập trên cùng một tập huấn luyện.
- Mỗi bộ phân loại có thể thuộc các loại khác nhau: SVM, Logistic Regression, Random Forest, K-Nearest Neighbors...
- Mỗi bộ phân loại đạt độ chính xác riêng lẻ, ví dụ khoảng 80%.
- Kỹ thuật: Tổng hợp các dự đoán của tất cả các bộ phân loại và đưa ra dự đoán tập hợp.

Huấn luyện các bộ phân loại đa dạng

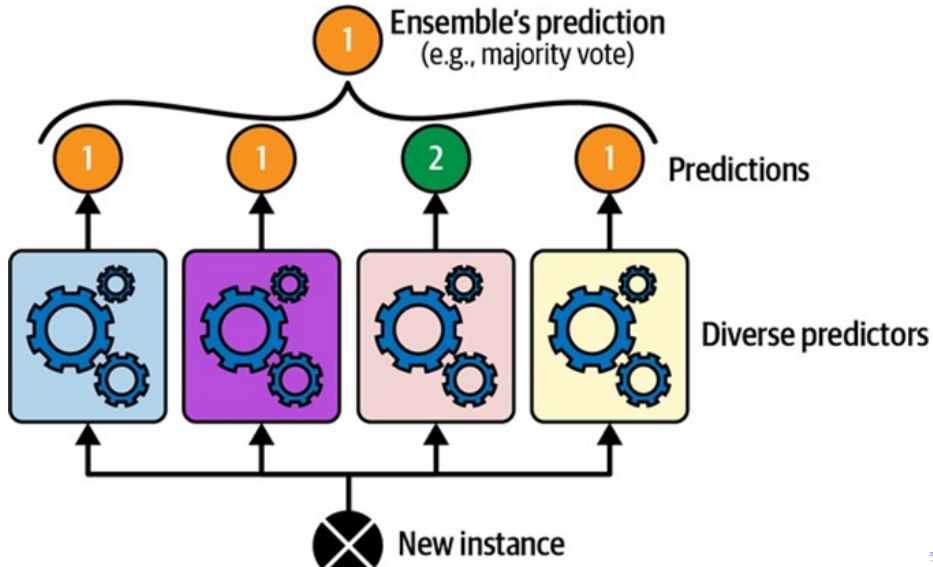


Hình 7-1. Huấn luyện các bộ phân loại đa dạng

Bộ phân loại bỏ phiếu cứng (Hard Voting)

- **Bỏ phiếu cứng (Hard voting):** Dự đoán của tập hợp là lớp nhận được nhiều phiếu bầu đa số nhất từ các bộ phân loại riêng lẻ.
- Mặc dù đơn giản, bộ phân loại bỏ phiếu cứng thường đạt được độ chính xác cao hơn bộ phân loại tốt nhất trong tập hợp.
- Ngay cả khi mỗi bộ phân loại chỉ là một **bộ học yếu** (weak learner - chỉ tốt hơn đoán ngẫu nhiên một chút), tập hợp vẫn có thể trở thành một **bộ học mạnh** (strong learner) nếu có đủ số lượng bộ học đa dạng.

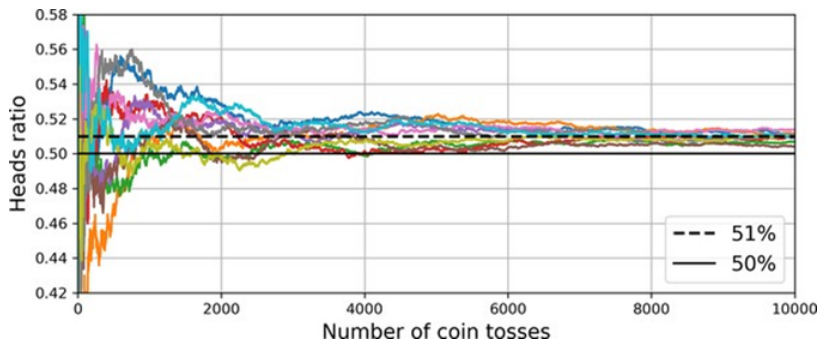
Dự đoán bỏ phiếu cứng



Nguyên lý đằng sau bộ phân loại biểu quyết

- Làm sao để các bộ học yếu kết hợp lại thành bộ học mạnh?
- **Phép tương tự:** Tung một đồng xu hơi lệch (51% sấp, 49% ngửa).
- Nếu tung 1.000 lần, tỷ lệ mặt sấp sẽ tiến gần 51%. Xác suất để đa số (trên 500 lần) là mặt sấp lên đến $\sim 75\%$.
- Tung 10.000 lần, xác suất đạt đa số mặt sấp tăng lên $> 97\%$.
- Đây là ứng dụng của **Luật số lớn (Law of Large Numbers)**.

Luật số lớn



Hình 7-3. Luật số lớn - Tỷ lệ mặt sấp khi số lần tung tăng

- *Lưu ý:* Tập hợp chỉ hoạt động hoàn hảo khi các bộ phân loại **hoàn toàn độc lập**, mắc lỗi không tương quan. Do cùng dùng một tập dữ liệu, chúng thường tương quan → làm giảm độ chính xác tập hợp so với kỳ vọng lý thuyết.

Mã nguồn: Bộ phân loại bỏ phiếu cứng (Phần 1)

Sử dụng VotingClassifier trong Scikit-Learn trên tập dữ liệu moons:

```
1 from sklearn.datasets import make_moons
2 from sklearn.ensemble import RandomForestClassifier, VotingClassifier
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.model_selection import train_test_split
5 from sklearn.svm import SVC
6
7 X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
8 X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
9
```

Mã nguồn: Bộ phân loại bỏ phiếu cứng (Phần 2)

Định nghĩa và huấn luyện tập hợp:

```
1 voting_clf = VotingClassifier(  
2     estimators=[  
3         ('lr', LogisticRegression(random_state=42)),  
4         ('rf', RandomForestClassifier(random_state=42)),  
5         ('svc', SVC(random_state=42))  
6     ]  
7 )  
8 voting_clf.fit(X_train, y_train)  
9  
10 # Kiểm tra kết quả:  
11 voting_clf.score(X_test, y_test) # Output: 0.912  
12
```

- Độ chính xác 91.2%, cao hơn độ chính xác của Logistic (86.4%), Random Forest (89.6%) và SVC (89.6%).

Bỏ phiếu mềm (Soft Voting)

- Nếu tất cả các bộ phân loại có khả năng ước tính xác suất lớp (có phương thức `predict_proba()`), tập hợp có thể đưa ra dự đoán dựa trên **trung bình xác suất** của từng lớp trên tất cả các bộ phân loại.
- **Bỏ phiếu mềm (Soft voting):** Chọn lớp có xác suất trung bình cao nhất.
- Thường đạt hiệu suất cao hơn bỏ phiếu cứng vì nó ưu tiên các phiếu bầu có độ tin cậy cao.

Mã nguồn: Bỏ phiếu mềm

Đặt siêu tham số `voting="soft"` và đảm bảo các mô hình (như SVC) có tính xác suất (`probability=True`):

```
1 voting_clf.voting = "soft"
2 voting_clf.named_estimators["svc"].probability = True
3 voting_clf.fit(X_train, y_train)
4 voting_clf.score(X_test, y_test)
5 # Output: 0.92
6
```

- Độ chính xác tăng lên 92% chỉ nhờ sử dụng bỏ phiếu mềm.

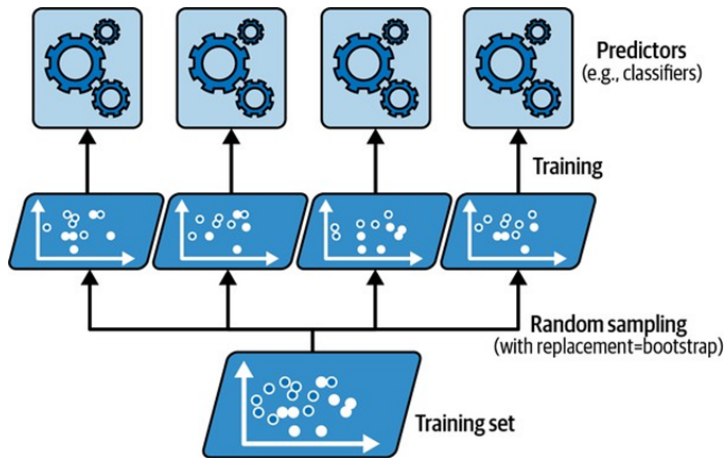
Túi hóa (Bagging) và Dán nhãn (Pasting)

- Thay vì dùng nhiều thuật toán khác nhau, một cách tiếp cận khác là sử dụng **cùng một thuật toán** cho mỗi bộ dữ liệu huấn luyện trên các **tập con ngẫu nhiên khác nhau** của tập huấn luyện.
- **Túi hóa (Bagging - Bootstrap Aggregating):** Lấy mẫu ngẫu nhiên *có hoàn lại* (có thay thế). Một mẫu có thể được chọn nhiều lần cho cùng một bộ dữ liệu huấn luyện.
- **Dán nhãn (Pasting):** Lấy mẫu ngẫu nhiên *không hoàn lại* (không thay thế). Một mẫu chỉ được chọn nhiều nhất một lần cho mỗi bộ dữ liệu huấn luyện.

Đặc điểm của Bagging và Pasting

- Hàm tổng hợp cuối cùng thường là **chế độ thống kê** (đối với phân loại) hoặc **giá trị trung bình** (đối với hồi quy).
- Mỗi bộ dự đoán riêng lẻ có **độ lệch (bias) cao hơn** so với khi huấn luyện trên toàn bộ dữ liệu.
- Nhưng kết hợp lại, tập hợp có độ lệch tương tự nhưng **phương sai (variance) thấp hơn** rất nhiều.
- **Mở rộng tốt:** Các bộ dự đoán độc lập nên có thể huấn luyện song song (tận dụng đa lõi CPU hoặc cụm máy chủ).

Minh họa: Quá trình lấy mẫu Bagging/Pasting



Hình 7-4. Túi hóa và Dán nhãn trên các mẫu ngẫu nhiên

Bagging và Pasting trong Scikit-Learn

- Scikit-Learn cung cấp lớp `BaggingClassifier` (hoặc `BaggingRegressor`).
- **Tham số quan trọng:**
 - `n_estimators`: Số lượng bộ dự đoán (ví dụ: 500 cây).
 - `max_samples`: Kích thước tập mẫu ngẫu nhiên cho mỗi bộ.
 - `bootstrap=True`: Sử dụng Bagging (hoặc `False` cho Pasting).
 - `n_jobs=-1`: Sử dụng tất cả các lõi CPU để tăng tốc.

Mã nguồn: BaggingClassifier

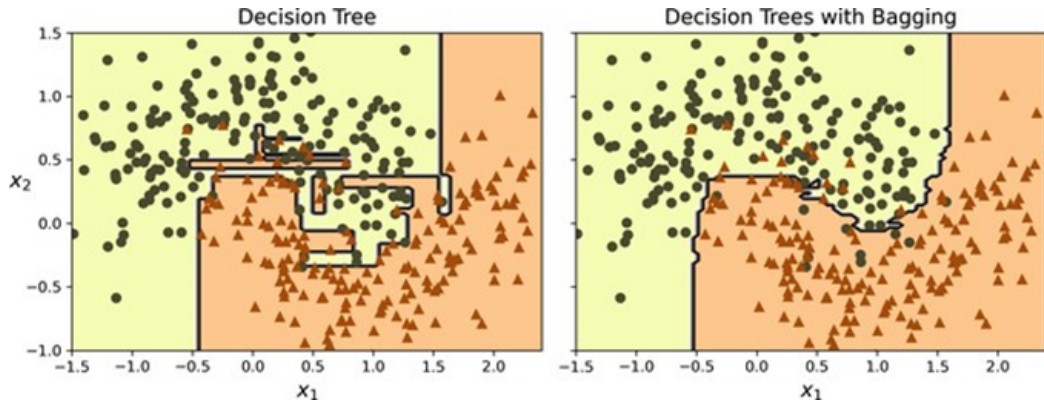
Huấn luyện tập hợp gồm 500 cây quyết định bằng Bagging:

```
1 from sklearn.ensemble import BaggingClassifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 bag_clf = BaggingClassifier(
5     DecisionTreeClassifier(),
6     n_estimators=500,
7     max_samples=100,
8     bootstrap=True, # Doi thanh False de dung Pasting
9     n_jobs=-1,
10    random_state=42
11 )
12 bag_clf.fit(X_train, y_train)
13
```

Phân tích kết quả của Bagging

- Bagging mang lại nhiều đa dạng hơn trong các tập con huấn luyện, do đó Bagging có độ lệch nhỏ hơn một chút so với Pasting.
- Sự đa dạng này giúp các bộ dự đoán ít tương quan hơn → **phương sai của tập hợp được giảm mạnh hơn.**
- Nhìn chung, Bagging thường mang lại mô hình tổng quát hóa tốt hơn và là lựa chọn ưu tiên mặc định.

So sánh Cây đơn lẻ và Tập hợp túi hóa



Hình 7-5. Cây quyết định đơn lẻ (trái) vs. Tập hợp Bagging 500 cây (phải)

Đánh giá ngoài mẫu (Out-of-Bag Evaluation)

- Trong Bagging, việc lấy mẫu có hoàn lại (bootstrap) khiến một số trường hợp được lấy nhiều lần, số khác thì không được lấy.
- Thống kê cho thấy: mỗi bộ dự đoán chỉ lấy trung bình khoảng **63%** số trường hợp.
- Khoảng **37%** trường hợp không bao giờ được bộ dự đoán đó nhìn thấy gọi là các trường hợp **ngoài mẫu (Out-of-Bag - OOB)**.
- Có thể dùng 37% OOB này để đánh giá (evaluate) tập hợp mà **không cần tập xác thực riêng biệt**.

Mã nguồn: Đánh giá OOB

Yêu cầu Scikit-Learn tính toán điểm OOB tự động bằng cách bật tham số
oob_score=True:

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(), n_estimators=500,  
3     oob_score=True, n_jobs=-1, random_state=42  
4 )  
5 bag_clf.fit(X_train, y_train)  
6  
7 # Xem diem OOB  
8 print(bag_clf.oob_score_) # Output: 0.896  
9  
10 # Kiểm tra trên tập Test thực tế  
11 from sklearn.metrics import accuracy_score  
12 y_pred = bag_clf.predict(X_test)  
13 print(accuracy_score(y_test, y_pred)) # Output: 0.92  
14
```

Random Patches và Random Subspaces

- Lớp BaggingClassifier hỗ trợ lấy mẫu theo **đặc trưng (features)** thông qua `max_features` và `bootstrap_features`.
- **Random Patches:** Lấy mẫu cả *trường hợp huấn luyện* và *đặc trưng*. Rất hiệu quả cho dữ liệu nhiều chiều (chẳng hạn như hình ảnh).
- **Random Subspaces:** Giữ toàn bộ trường hợp huấn luyện (`bootstrap=False`, `max_samples=1.0`) nhưng *chỉ lấy mẫu ngẫu nhiên đặc trưng*.
- Đổi một chút độ lệch cao hơn để lấy phương sai thấp hơn.

Rừng ngẫu nhiên (Random Forests)

- Rừng ngẫu nhiên là một tập hợp các cây quyết định, thường được huấn luyện bằng phương pháp **Bagging** với `max_samples` bằng toàn bộ kích thước tập huấn luyện.
- Thuật toán Rừng ngẫu nhiên **đưa thêm sự ngẫu nhiên** vào việc phát triển cây: thay vì tìm kiếm đặc trưng tốt nhất trong toàn bộ đặc trưng, nó tìm kiếm **đặc trưng tốt nhất trong một tập con ngẫu nhiên** của các đặc trưng (mặc định là $\sqrt{n}$ đặc trưng).
- Dẫn đến sự đa dạng cây lớn hơn, đổi độ lệch cao hơn lấy phương sai thấp hơn, mang lại một mô hình tổng quát hóa tốt.

Mã nguồn: RandomForestClassifier

Sử dụng lớp RandomForestClassifier tiện lợi và tối ưu hơn so với dùng BaggingClassifier lồng DecisionTreeClassifier:

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rnd_clf = RandomForestClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 rnd_clf.fit(X_train, y_train)
10 y_pred_rf = rnd_clf.predict(X_test)
11
```

So sánh BaggingClassifier và RandomForestClassifier

Mã lệnh BaggingClassifier sau đây gần tương đương với RandomForestClassifier trước đó:

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),  
3     n_estimators=500,  
4     n_jobs=-1,  
5     random_state=42  
6 )  
7
```

- RandomForestClassifier được tối ưu hiệu năng tốt hơn nhiều cho cây quyết định.

Cây ngẫu nhiên cực đại (Extra-Trees)

- Có thể làm cho cây ngẫu nhiên hơn nữa bằng cách sử dụng các **ngưỡng ngẫu nhiên** cho mỗi đặc trưng thay vì tìm ngưỡng chia cắt tốt nhất có thể.
- Tập hợp gồm các cây cực kỳ ngẫu nhiên này được gọi là **Extra-Trees** (Extremely Randomized Trees).
- Lợi ích: Tăng tốc độ huấn luyện đáng kể do không phải tốn kém thời gian tìm ngưỡng tốt nhất, đổi thêm một chút độ lệch lấy phương sai thấp hơn nữa.

Cài đặt Extra-Trees trong Scikit-Learn

Trong `DecisionTreeClassifier`, đặt tham số `splitter="random"` để tạo cây Extra-Tree đơn lẻ. Hoặc sử dụng tập hợp trực tiếp bằng `ExtraTreesClassifier`:

```
1 from sklearn.ensemble import ExtraTreesClassifier
2
3 extra_clf = ExtraTreesClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 # API giống hệt với RandomForestClassifier
10
```

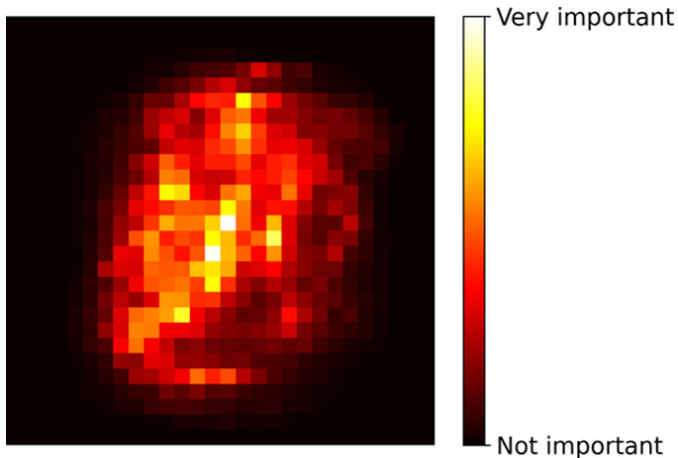
Tầm quan trọng của đặc trưng

- **Feature Importances:** Rừng ngẫu nhiên rất tuyệt vời để đo lường tầm quan trọng tương đối của từng đặc trưng.
- Scikit-Learn đo lường bằng cách tính toán mức độ mà các nút cây sử dụng một đặc trưng để **làm giảm độ không tinh khiết** (impurity), tính trung bình trên toàn bộ rừng.
- Kết quả được chuẩn hóa để tổng độ quan trọng của tất cả các đặc trưng bằng 1.
- Thuộc tính chứa kết quả: `feature_importances_`.

Mã nguồn: Tính toán Feature Importances

```
1 from sklearn.datasets import load_iris
2 iris = load_iris(as_frame=True)
3 rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
4 rnd_clf.fit(iris.data, iris.target)
5
6 for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
7     print(round(score, 2), name)
8
9 # Output:
10 # 0.11 sepal length (cm)
11 # 0.02 sepal width (cm)
12 # 0.44 petal length (cm)    <-- Quan trọng nhất
13 # 0.42 petal width (cm)    <-- Quan trọng thứ hai
14
```

Tầm quan trọng pixel MNIST



Hình 7-6. Tầm quan trọng của pixel trên ảnh MNIST

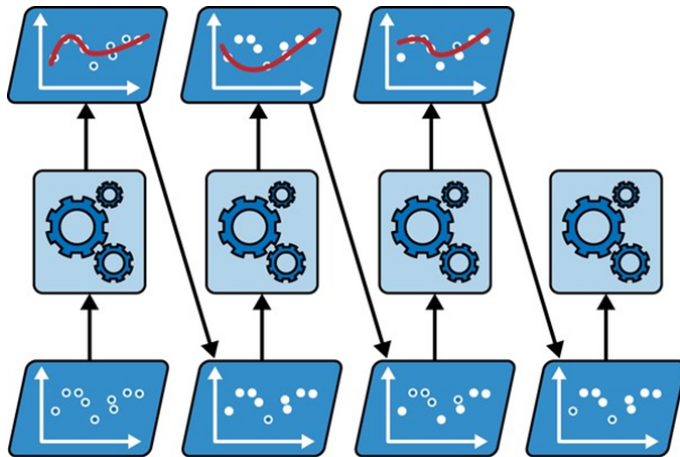
Tăng cường (Boosting)

- **Boosting** (Tăng cường giả thuyết) là phương pháp tập hợp giúp biến các bộ học yếu thành bộ học mạnh.
- Ý tưởng cốt lõi: Huấn luyện các bộ dự đoán **theo trình tự**, mỗi bộ dự đoán sau sẽ cố gắng **sửa chữa lỗi** của bộ tiền nhiệm.
- Hai phương pháp phổ biến nhất:
 - 1 AdaBoost (Adaptive Boosting)
 - 2 Gradient Boosting

AdaBoost (Adaptive Boosting)

- Cách bộ dự đoán mới sửa lỗi: **Tập trung nhiều hơn vào các trường hợp huấn luyện bị phân loại sai** bởi bộ tiền nhiệm.
- Quy trình:
 - 1 Huấn luyện mô hình cơ sở (thường là Cây quyết định một nút - decision stump) và dự đoán.
 - 2 **Tăng trọng số** tương đối của các trường hợp bị sai.
 - 3 Huấn luyện mô hình thứ 2 dựa trên trọng số mới cập nhật, rồi lại dự đoán và tăng trọng số, v.v.
- Kỹ thuật này khá giống Gradient Descent, nhưng thay vì chỉnh tham số cho một mô hình, nó thêm mô hình mới vào tập hợp.

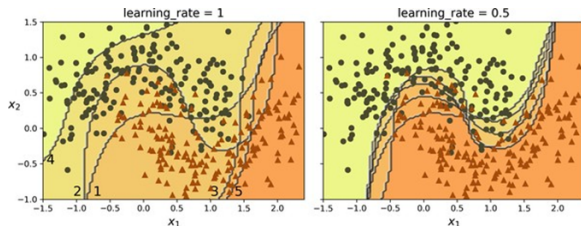
Cập nhật trọng số trong AdaBoost



Hình 7-7. Huấn luyện AdaBoost tuần tự với cập nhật trọng số

Phân tích trọng số trong SVM

- Hình bên minh họa 5 bộ SVM (RBF kernel) liên tiếp trên dữ liệu moons.
- Biểu đồ trái: Learning rate lớn, bộ thứ 2 sửa rất mạnh các lỗi của bộ đầu.
- Biểu đồ phải: Learning rate bằng một nửa, các SVM sửa đổi một cách cẩn thận và dần dần hơn.



Toán học AdaBoost: Tỷ lệ lỗi và Trọng số bộ dự đoán

- Trọng số ban đầu của mỗi trường hợp $w^{(i)} = 1/m$.
- **Tỷ lệ lỗi có trọng số của bộ thứ j (r_j):**

$$r_j = \frac{\sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)}}^m w^{(i)}}{\sum_{i=1}^m w^{(i)}} \quad (1)$$

- **Trọng số của bộ dự đoán thứ j (α_j):**

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j} \quad (2)$$

Trong đó η là learning rate (mặc định 1). Bộ càng chính xác thì α_j càng cao. Nếu đoán bừa thì $\alpha_j \approx 0$.

- Quy tắc cập nhật trọng số trường hợp ($w^{(i)}$):

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{nếu } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{nếu } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases} \quad (3)$$

- Sau đó, tất cả trọng số được chuẩn hóa (chia cho $\sum w^{(i)}$).
- Dự đoán cuối cùng:

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{j=1, \hat{y}_j(\mathbf{x})=k}^N \alpha_j \quad (4)$$

Mã nguồn: AdaBoostClassifier

Scikit-Learn sử dụng biến thể đa lớp tên là SAMME (hoặc SAMME.R để tận dụng xác suất).

```
1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 ada_clf = AdaBoostClassifier(
5     DecisionTreeClassifier(max_depth=1), # Cây 1 nút (decision stump)
6     n_estimators=30,
7     learning_rate=0.5,
8     random_state=42
9 )
10 ada_clf.fit(X_train, y_train)
11
```

Gradient Boosting

- Tương tự AdaBoost: huấn luyện bộ sau sửa lỗi của bộ trước.
- Khác biệt: Thay vì thay đổi trọng số trường hợp, Gradient Boosting **khớp bộ dự đoán mới với các lỗi còn lại (residual errors)** do bộ dự đoán trước tạo ra.
- Gradient Tree Boosting (GBRT): Sử dụng Cây quyết định làm bộ cơ sở.

Mã nguồn: Gradient Tree Boosting thủ công (Phần 1)

Tạo dữ liệu và huấn luyện cây đầu tiên:

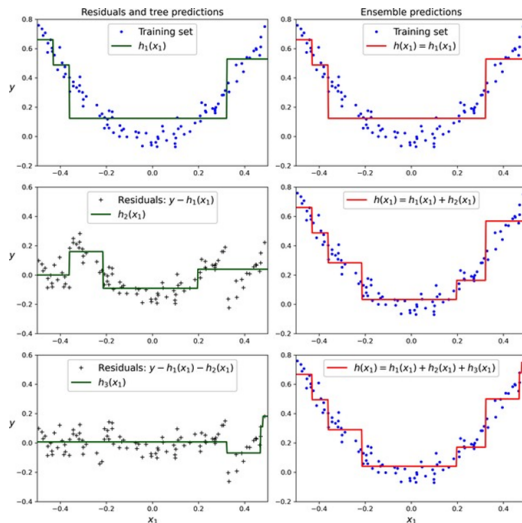
```
1 from sklearn.tree import DecisionTreeRegressor
2 import numpy as np
3
4 np.random.seed(42)
5 X = np.random.rand(100, 1) - 0.5
6 y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100)
7
8 tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
9 tree_reg1.fit(X, y)
10
```

Mã nguồn: Gradient Tree Boosting thủ công (Phần 2)

Huấn luyện trên phần dư (residuals) và kết hợp:

```
1 y2 = y - tree_reg1.predict(X) # Phan du 1
2 tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
3 tree_reg2.fit(X, y2)
4
5 y3 = y2 - tree_reg2.predict(X) # Phan du 2
6 tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
7 tree_reg3.fit(X, y3)
8
9 # Du doan moi bang tong cac cay:
10 X_new = np.array([[-0.4], [0.], [0.5]])
11 y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
12
```

Quá trình huấn luyện GBRT



Hình 7-9. Các dự đoán của từng cây (trái) và tổng hợp của tập hợp (phải)

GradientBoostingRegressor trong Scikit-Learn

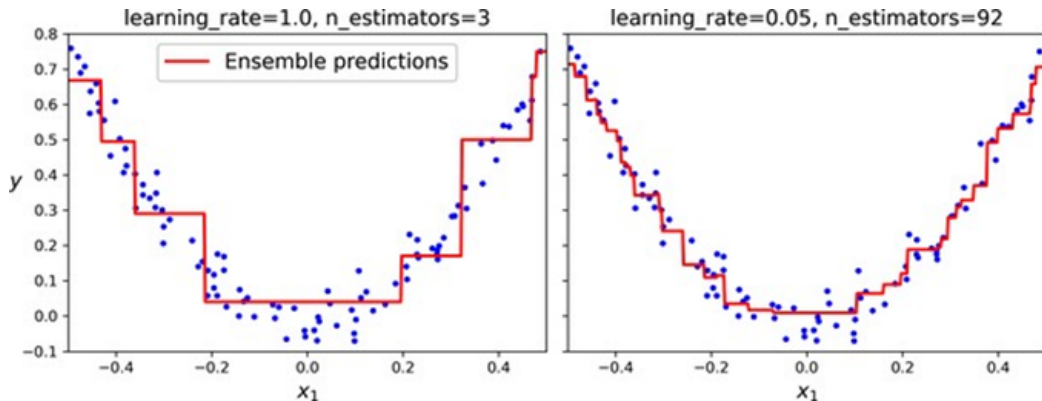
Sử dụng lớp có sẵn để tự động quá trình này:

```
1 from sklearn.ensemble import GradientBoostingRegressor
2
3 gbrt = GradientBoostingRegressor(
4     max_depth=2,
5     n_estimators=3,
6     learning_rate=1.0,
7     random_state=42
8 )
9 gbrt.fit(X, y)
10
```

Vấn đề quá khớp và Shrinkage

- `learning_rate`: Kiểm soát mức độ đóng góp của từng cây.
- Nếu đặt giá trị thấp (ví dụ 0.05), sẽ cần nhiều cây hơn để khớp dữ liệu (Shrinkage - một phương pháp chính quy hóa).
- Nếu số lượng cây `n_estimators` quá ít → Dưới khớp.
- Nếu số lượng cây `n_estimators` quá nhiều → Quá khớp.

Các tập hợp GBRT không đủ và vừa đủ cây



Hình 7-10. Dưới khớp (trái) vs. Vừa vặn (phải)

Dừng sớm (Early Stopping) trong GBRT

Tránh phải dùng GridSearchCV dò `n_estimators`, ta có thể dùng Dừng sớm qua `n_iter_no_change`:

```
1 gbrt_best = GradientBoostingRegressor(  
2     max_depth=2,  
3     learning_rate=0.05,  
4     n_estimators=500,  
5     n_iter_no_change=10, # Dừng nếu 10 lần lặp liên tiếp không giảm lỗi  
6     random_state=42  
7 )  
8 gbrt_best.fit(X, y)  
9  
10 print(gbrt_best.n_estimators_) # Chỉ cần 92 cây (tu dừng sớm) thay vì 500  
11
```

Tăng cường Gradient dựa trên Biểu đồ (HGB)

- GBRT chậm trên dữ liệu lớn. Scikit-Learn cung cấp **Histogram-Based Gradient Boosting (HGB)** tối ưu cho dữ liệu hàng triệu dòng.
- **Cơ chế:** Nhóm các đặc trưng lại (binning) thành tối đa 255 nhóm (bins). Giảm đáng kể số ngưỡng phân chia.
- **Độ phức tạp:** Giảm từ $O(b \times m \times \log(m))$ xuống $O(b \times m)$, huấn luyện nhanh hơn hàng trăm lần.
- HGB xử lý luôn được cả Missing values và Categorical Features (không cần Imputer hay OneHotEncoder).

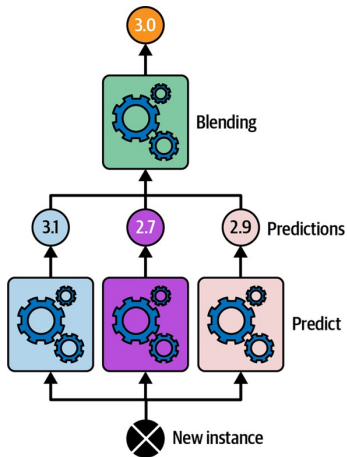
Mã nguồn: HistGradientBoostingRegressor

```
1 from sklearn.ensemble import HistGradientBoostingRegressor
2 from sklearn.pipeline import make_pipeline
3 from sklearn.compose import make_column_transformer
4 from sklearn.preprocessing import OrdinalEncoder
5
6 hgb_reg = make_pipeline(
7     make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
8                             remainder="passthrough"),
9     HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
10 )
11 hgb_reg.fit(housing, housing_labels)
```

Phân lớp xếp chồng (Stacking)

- **Stacking (Stacked Generalization):** Thay vì dùng hàm tầm thường như Bỏ phiếu đa số, tại sao không **huấn luyện một mô hình máy học khác** để tổng hợp các dự đoán?
- Bộ mô hình lớp cuối cùng được gọi là **bộ trộn (blender)** hay meta-learner.
- Lấy các dự đoán của các bộ học cơ sở làm đầu vào (input) và học cách tạo ra dự đoán chính xác nhất từ đó.

Tổng hợp dự đoán bằng bộ trộn

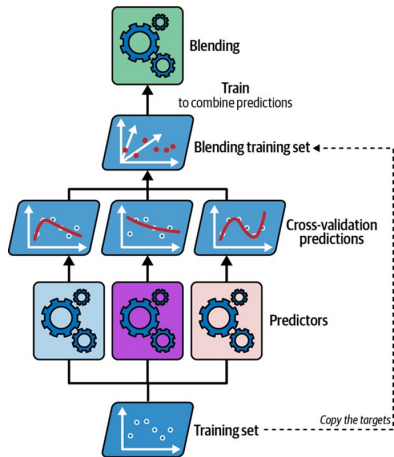


Hình 7-11. Sử dụng bộ trộn để kết hợp 3 mô hình

Khái niệm bộ trộn (Blender)

- Để huấn luyện bộ trộn, ta chia dữ liệu thành 2 phần:
- **Tập 1 (Tập huấn luyện):** Dùng để huấn luyện các mô hình cơ sở.
- **Tập 2 (Tập pha trộn - Blend/Hold-out):** Các mô hình cơ sở sẽ dự đoán trên tập này. Các dự đoán xuất ra sẽ đóng vai trò là **đặc trưng (features) mới**. Bộ trộn sẽ được huấn luyện trên tập đặc trưng mới này kết hợp với nhãn gốc.
- Scikit-Learn sử dụng `cross_val_predict` (K-Fold) để tránh làm lãng phí dữ liệu.

Huấn luyện bộ trộn trong tập hợp

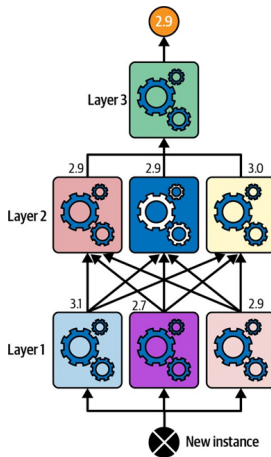


Hình 7-12. Tạo dữ liệu huấn luyện cho bộ trộn qua Out-of-Sample Predictions

Kiến trúc xếp chồng phức tạp

- Không dừng lại ở 1 bộ trộn, ta có thể huấn luyện **nhiều bộ trộn khác nhau** (ví dụ: Linear, Random Forest) trên cùng một layer.
- Sau đó thêm 1 bộ trộn Tối cao lên trên cùng (Layer 3) để đưa ra dự đoán cuối.
- Càng nhiều lớp, mô hình càng tốn kém thời gian huấn luyện và độ phức tạp tính toán.

Tập hợp xếp chồng đa lớp



Hình 7-13. Mạng Stacking đa tầng (Multi-layer Stacking Ensemble)

Cài đặt StackingClassifier trong Scikit-Learn

```
1 from sklearn.ensemble import StackingClassifier
2
3 stacking_clf = StackingClassifier(
4     estimators=[
5         ('lr', LogisticRegression(random_state=42)),
6         ('rf', RandomForestClassifier(random_state=42)),
7         ('svc', SVC(probability=True, random_state=42))
8     ],
9     final_estimator=RandomForestClassifier(random_state=43),
10    cv=5 # 5-Fold cross validation
11 )
12 stacking_clf.fit(X_train, y_train)
13
```


Tổng kết Chương 7

- **Học tổ hợp (Ensemble Learning)** là công cụ quyền lực đầy hiệu năng mô hình đến giới hạn.
- **Voting & Bagging/Pasting:** Đơn giản, dễ song song hóa, giảm phương sai cực tốt. Rừng ngẫu nhiên (Random Forest) là thuật toán tiêu biểu, hiệu năng ổn định.
- **Boosting (AdaBoost, GBRT, HGB):** Tập trung sửa sai, chuyển bộ học yếu thành mạnh. HGB là đột phá về tốc độ cho dữ liệu lớn.
- **Stacking:** Đẩy mạnh sức mạnh thông qua mô hình Meta-learner thay vì trung bình cứng nhắc.
- *Rừng ngẫu nhiên và GBRT luôn là những mô hình đầu tiên nên thử nghiệm trong mọi dự án Machine Learning dạng bảng.*

CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE!

Q & A